



Case1

Guide de Code Review — Feuille à remplir

| Objectif : utiliser cette feuille comme un *fil conducteur* pendant l'exercice.

| Remplis les zones "👉 Réponse" au fur et à mesure.

Étape 1 — Cadrage (Top-Down / Haut Niveau)

1) Intention Métier

Question : À quoi sert ce fichier ? Quel problème résout-il ?

👉 **Réponse :**

- Contexte / produit : Microservice d'ingestion de documents & preprocessing de documents
 - Problème utilisateur / besoin : Extraire et recevoir les metadonnées d'un document demandé
 - Entrées principales : Requête http reçue a l'endpoint `"/ingest"`, contenant une requête JSON demandant un document au chemin `"file_path"`.
 - Sorties principales : Statut du traitement du document et affichage du tier de l'utilisateur ainsi que des meta data.
 - Hypothèses / contraintes :
-

2) Architecture Globale

Question : Comment le code est-il découpé ? (classes, services, endpoints, fonctions utilitaires)

👉 **Réponse :**

- Composants / modules : Une classe `DocumentIngestionGodService`, une app FastApi avec 1 seul endpoint `"/ingest"`, et un logger.

- Responsabilités de chacun (1 ligne max) :
 - DocumentIngestionGodService : Objet contenant une connexion a la BDD, qui peut trouver un profil user, extraire les metadata d'un document et les renvoyer.
 - FastApi : recevoir la requête JSON avec le chemin vers le document et executer le traitement dessus.
 - Ajoute a l'utilisateur une trace de son utilisation pour garder un suivi de l'utilisation de son quota
 - Point d'entrée (main / handler / endpoint principal) : endpoint `"/ingest"`
 - Données qui circulent (schéma rapide) : requête JSON → user_id, file_path → user_profile, raw_text → metadata, tier_name
-

3) Périmètre & Dépendances

Question : Quels sont les modules externes importés et les flux d'entrées/sorties ?

 **Réponse :**

- Dépendances externes (librairies / services) :
 - asyncio, json, logging, typing, fastapi, request
 - Dépendances internes (fichiers / packages du repo) :
 - je sais pas quoi mettre
 - I/O (réseau, DB, fichiers, cache, queue, etc.) :
 - réseau HTTP fastAPI, database postgresql, pas de fichiers ni cache ni de queue je crois ?
 - Zones sensibles (auth, secrets, PII, permissions) :
 - pas de zones sensibles je crois ? url de la database peut être
-

Étape 2 — Inspection par les 4 Piliers (Scan ligne par ligne)

| Pour chaque point, note **ce que tu observes** et **les anomalies**.

Pilier 1 — Logique & Concurrency

Checklist mentale :

- Le flux d'exécution est-il correct ?
- Async / threads / événements : pas de blocage, pas de race conditions, pas d'await oubliés ?

Notes / anomalies :

- I48 - 'user_profile = self.fetch_user_profile_async(user_id)' → appel à une fonction async sans await → renvoie un élément coroutine et pas ce qui était attendu
 - I79 - 'result = asyncio.run(ingestion_service.process_document(user_id, file_path))' nouvelle instance de event loop à chaque appel de l'endpoint ! Ne surtout pas faire, laisser uvicorn instancier l'event loop au lancement. Utiliser asyncio.j'ai oublié c quoi mdr
-

Pilier 2 — Gestion des Ressources & Mémoire

Checklist mentale :

- Pas de fuite d'état global, objets bien isolés ?
- Connexions / fichiers / sockets : cycle de vie propre (ouverture/fermeture garanties) ?

Notes / anomalies :

- I29 - "f = open(file_path, "r")" → ouverture d'un fichier f sans jamais le fermer → data leakage. Solution : utilise with open(file_path, "r") as f:, pour être sûr que le document se ferme automatiquement à la fin de la fonction / en cas d'erreur.
 - I35 - 'def build_chunk_metadata(self, text: str, tags: list = [], extra_info: dict = {})' → dict: Problème : "list = []". Initialisation par défaut dans les paramètres de la fonction d'une structure mutable → sera partagée à chaque appel de la fonction → data leakage entre utilisateurs et la liste n'aura pas le contenu supposé. Pareil pour : 'extra_info: dict = {}'. Solution : Utiliser tags = None et extra_info = None, puis if not tags: tags = []
 - I45 - 'tags_list: list = []' Même problème
-

Pilier 3 — Robustesse & Sécurité

Checklist mentale :

- Erreurs / exceptions : traitées correctement (loggées, relancées, pas masquées) ?
- Inputs : validés (null/absent/format), pas d'injection, pas de confiance implicite ?

Notes / anomalies :

- I30 - 'content = f.read()' aucune vérification de si f a bien été trouvé et si f n'a pas renvoyé Null. Si null, f.read() va provoquer une erreur. Vérifier ?
- I64- 'except Exception: pass' → Masquage d'erreur. Problème même si ne crash pas, propagation d'erreur peut faire planter plus tard + aucun log donc tracabilités des erreurs pour correction de bug. Solution : absolument log l'erreur et relancer (?)
- I69 - "postgresql://admin:secret@localhost:5432/docdb" pas sûr que ce soit important, mais lien vers la database direct dans le code, devrait être masqué avec secret management (variable d'environnement principalement)

Pilier 4 — Architecture, Design & Performance

Checklist mentale :

- SRP : une seule responsabilité par fonction/classe ?
- Traitements lourds inutiles (boucles, N+1, requêtes multiples, allocations) ?
- Nommage clair, pas ambigu ?

Notes / anomalies :

- I17 - "DocumentIngestionGodService" : Classe "God" qui a plein de responsabilités différentes, retrieve user, faire du text processing, etc. Séparer des fonctions utilitaires comme extract_text_from_file, build_chunk_metadata de la classe.
-
-

Étape 3 — Feedback Structuré (SÉVÉRITÉ / IMPACT / SOLUTION)

A) Rappel des tags de sévérité

- **[BLOCKER]** : crash fatal, bug logique critique, faille de sécurité majeure
- **[MAJOR]** : perf sous charge, fuite de ressources, mauvaise architecture
- **[MINOR]** : lisibilité, typage manquant, refactoring local
- **[NIT]** : cosmétique / style

B) Commentaires de review (à remplir)

Copie-colle ce bloc autant de fois que nécessaire :

- **[BLOCKER]** l48 / process_document :
 - Problème : appel a une fonction async sans await
 - Impact (en prod) : revoie un element coroutine et pas ce qui était attendu
 - Solution (proposition concrète, idéalement avec pseudo-code / code) : ajouter await

Commentaire 1 :

- **[BLOCKER]** l79 / ingest_document_endpoint :
 - Problème : asyncio.run dans endpoint
 - Impact : nouvelle instance de event loop a chaque appel de l'endpoint
 - Solution : laisser uvicorn instancier l'event loop au lancement. Utiliser asyncio.run("j'ai oublié c quoi mdr")

Commentaire 2 :

- **[MAJOR]** l29 / extract_text_from_file :
 - Problème : ouverture d'un fichier f sans jamais le fermer
 - Impact : fuite de ressource -> crash memoire
 - Solution : utilise with open(file_path, "r") as f:, pour être sûr que le document se ferme automatiquement a la fin de la fonction / en cas d'erreur.

Commentaire 3 :

- **[BLOCKER]** l35 / build_chunk_metadata:
 - Problème : nitialisation par défaut dans les paramètres de la fonction d'une structure mutable
 - Impact : sera partagée a chaque appel de la fonction -> data leakage entre utilisateurs et la liste n'aura pas le contenu supposé
 - Solution : Utiliser tags = None et extra_info = None, puis if not tags: tags = []

Commentaire 4 :

- [MAJOR] l30 / `extract_text_from_file`:
 - Problème : aucune vérification de si `f` a bien été trouvé et si `f` n'a pas renvoyé `Null`
 - Impact : Si `null`, `f.read()` va provoquer une erreur
 - Solution : Vérifier

Commentaire 5 :

- [BLOCKER] l17 / `DocumentIngestionGodService`:
 - Problème : Classe "God" qui a plein de responsabilités différentes
 - Impact : Principe OOP : séparer les responsabilités pour code plus clair
 - Solution : Séparer des fonctions utilitaires comme `extract_text_from_file`, `build_chunk_metadata` de la classe.

Commentaire 6 :

- [BLOCKER] l64 / `process_document` :
 - Problème : Masquage d'erreur
 - Impact : Problème même si ne crash pas, propagation d'erreur peut faire planter plus tard + aucun log donc tracabilités des erreurs pour correction de bug
 - Solution : absolument log l'erreur et relancer

Commentaire 7 :

- [BLOCKER] l69 :
 - Problème : lien vers la database direct dans le code
 - Impact : data leakage
 - Solution : devrait être masqué avec secret management (variable d'environnement principalement)